FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 13 July 2016 - Part Theory

NAME (capital letters pls.):

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the two-letter alphabet $\Sigma = \{ a, b \}$. Answer the following questions:

   (a) Design a (preferably minimal) finite-state automaton A_1 that accepts all and only the strings of odd length over the alphabet Σ.

   (b) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A_2 equivalent to the regular expression R below:

   $$R = (a \mid b)^* \, b$$

   (c) In a systematic way, obtain the finite-state automaton A cartesian product of the previous automata A_1 and A_2, i.e., $A = A_1 \times A_2$.

   (d) If necessary, minimize the automaton A previously obtained.

   (e) (optional) From the minimal automaton A and using the BMC method (node elimination method), derive an equivalent regular expression R'. If necessary, rewrite this regular expression R' in order to simplify it and to highlight the characteristic properties of the language generated by R'.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language with round open and closed parentheses '(' and ')', generated by the well known *BNF* unambiguous grammar G below (axiom S):

   $$G\colon S \to (\ S\ )\ S \ \mid\ \varepsilon$$

   Answer the following questions:

   (a) Write a grammar G_1, *BNF* and unambiguous, that generates all and only the Dyck strings of language $L(G)$ whose substrings of consecutive open parentheses have an even length (0, 2, 4, ...).

   Examples:

   ε $\qquad$ $\overbrace{(\,(\,)\,)}^{2}$ $\qquad$ $(\overbrace{(\,)}^{2}\ \overbrace{(\,(\,)\,)}^{2}\,)$

   Counterexamples:

   $\overbrace{(\quad)}^{1}$ $\qquad$ $(\overbrace{(\,)}^{2}\ \overbrace{(\,(\,(\,)\,)\,)}^{3}\,)$

   To test grammar G_1, draw the syntax trees of the three above examples; also try to draw the syntax trees of the two counterexamples and shortly explain why it is impossible to complete the trees by means of grammar G_1.

   (b) (optional) Write a grammar G_2, *BNF* and unambiguous, that generates all and only the Dyck strings of language $L(G)$ whose substrings of consecutive closed parentheses have an odd length (1, 3, 5, ...).

   Examples:

   $(\overbrace{\quad}^{1})$ $\qquad$ $(\,(\overbrace{\quad}^{1})\ \overbrace{(\,)\,}^{3}\,)$

   Counterexamples:

   ε $\qquad$ $(\,(\overbrace{\quad}^{2})\,)$ $\qquad$ $\overbrace{(\quad)}^{1}\ (\overbrace{\quad}^{2})$

   To test grammar G_2, draw the syntax trees of the two above examples; also try to draw the syntax trees of the three counterexamples and shortly explain why it is impossible to complete the trees by means of grammar G_2.

---

2. Consider a language designed to describe a sport game organized by elimination tournaments, where at each stage of the game only the winners will play in the next stage, until exactly one competitor becomes the final winner.

   - In each stage, a player can be either a new team or the winner team of a previous stage, at any depth level.
   - Every match has two players and these attributes:
     - an associated date, defined by a number for the year, a string for the month and a number for the day
     - a scheduled time, defined by two numbers for the hour and the minute
     - and a location, defined by a string

     For simplicity, assume that in the grammar all the numbers and strings are schematized by the terminals $\boldsymbol{n}$ and $\boldsymbol{s}$, respectively.
   - A document (i.e., a string of the language) consists of a non-empty list of tournament descriptions.

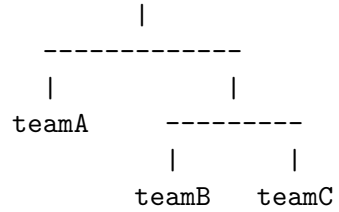
   Answer the following questions:

   (a) Write a grammar G, *EBNF* and unambiguous, that models the language described above.

   (b) Briefly illustrate how your grammar G can be modified in a systematic way to put a limit k on the height of the tree that represents the matches played in the tournament; exemplify the modification for the case $k = 3$.

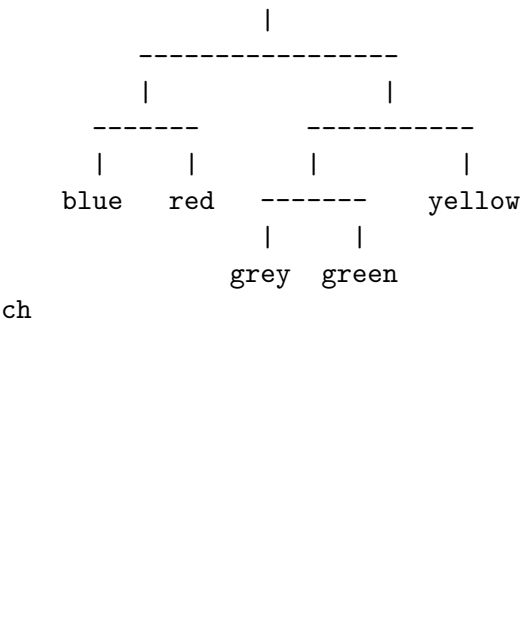
   Two sample language fragments are reported below. On the right side of each sample it is depicted a tree that represents the matches of the tournament.

```
tournament julyFun is
   match
      date 2016 july 29;
      time 21 : 30;
      location Rome;
      player1 is teamA;
      player2 is winner of match
         date 2016 july 23;
         time 17 : 00;
         location Milan;
         player1 is teamB;
         player2 is teamC;
      end match;
   end match;
end tournament julyFun;
```



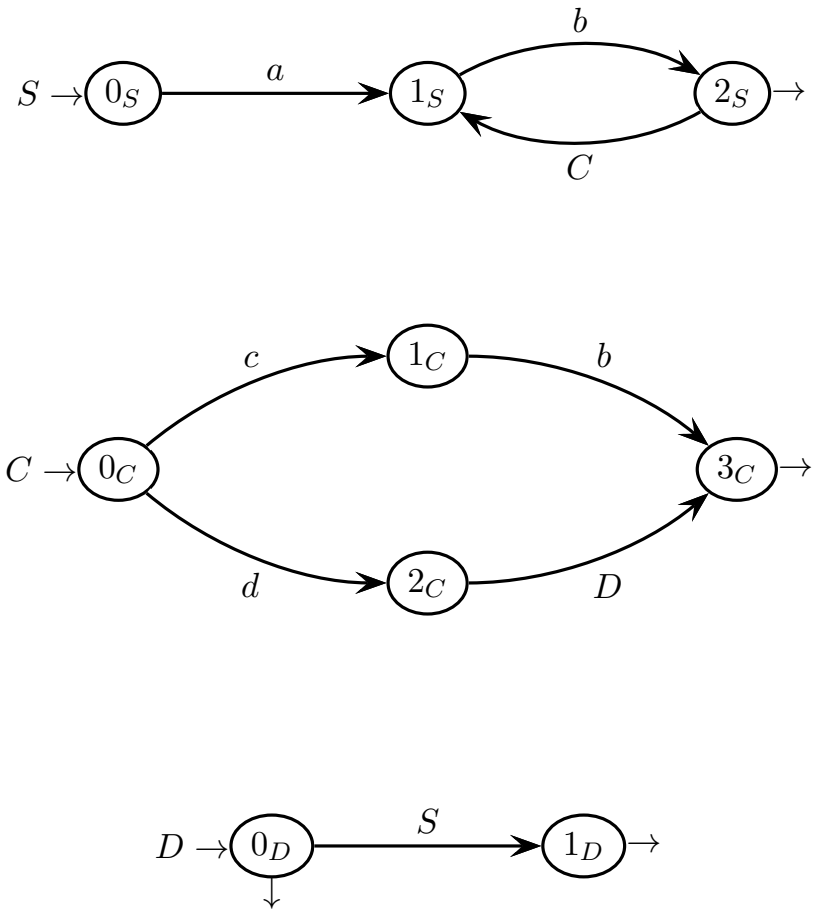
See the next page for one more example.

```
tournament springOutdoor is
   match
      ...
      player1 is winner of match
         ...
         player1 is blue
         player2 is red;
      end match;
      player2 is winner of match
         ...
         player1 is winner of match
            ...
            player1 is grey;
            player2 is green;
         end match;
         player2 is yellow;
      end match;
   end match;
end tournament springOutdoor;
```



3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G, represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the three-letter nonterminal alphabet $V = \{ S, C, D \}$ (axiom S).



Answer the following questions:

(a) Draw the complete pilot of grammar G, say if grammar G is of type $ELR(1)$ and shortly justify your answer.

(b) Write all the guide sets on the arcs of the machine net (shift and call arcs, and final arrows), say if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer.

(c) (optional) Write two (of three) syntactic procedures of the recursive descent syntax analyzer of grammar G.

---

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G_s (axiom S), for a language over the three-letter terminal alphabet $\{\ a,\ b,\ c\ \}$ and the two-letter nonterminal alphabet $\{\ S,\ A\ \}$:

$$G_s \begin{cases} S & \rightarrow & A\ c\ S \\ S & \rightarrow & A \\ A & \rightarrow & a\ a\ A\ b\ b \\ A & \rightarrow & c\ b \end{cases}$$

Grammar G_s generates strings composed of a sequence of groups of letters separated by letters c, like for instance string:

$$a^4\ c\ b^5\ c\ a^2\ c\ b^3$$

where each group consists of an *even* number (say $2k$ with $k \geq 0$) of letters a immediately followed by one letter c and then by $2k+1$ letters b.

(a) Write a syntactic translation scheme G_τ (or a translation grammar) that computes the following translation τ:

$$\tau\,(a^{2k_1}\ c\ b^{2k_1+1}\ c\ \ldots\ c\ a^{2k_n}\ c\ b^{2k_n+1}) = b^{2k_1+1}\ d\ a^{2k_1}\ e\ \ldots\ e\ b^{2k_n+1}\ d\ a^{2k_n}\ f$$

over the five-letter target alphabet $\{\ a,\ b,\ d,\ e,\ f\ \}$, with $k_i \geq 0$ and $1 \leq i \leq n$. Example:

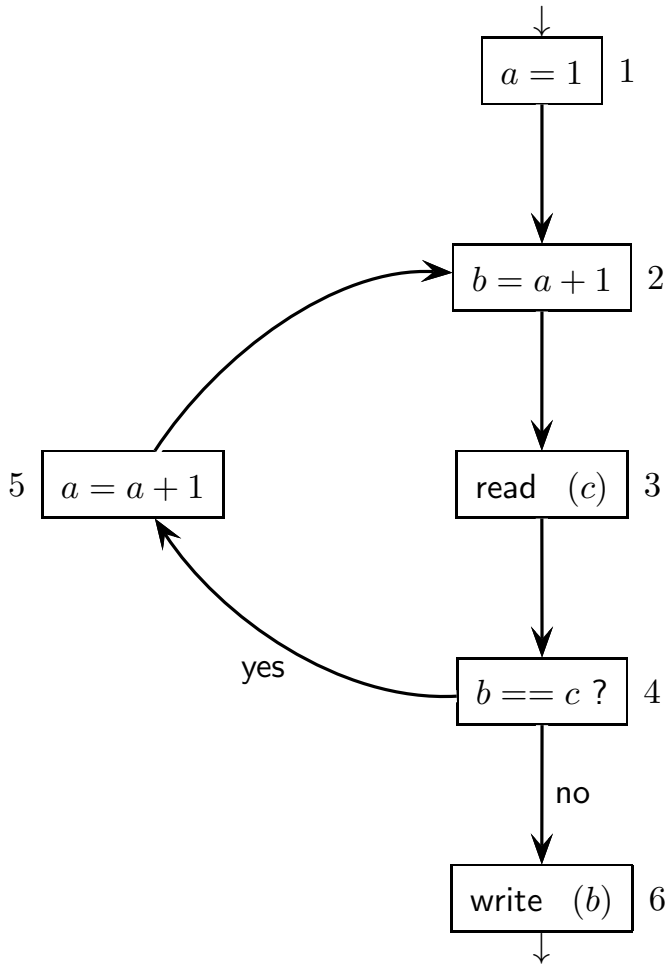
$$\tau\,(a^4\ c\ b^5\ c\ a^2\ c\ b^3) = b^5\ d\ a^4\ e\ b^3\ d\ a^2\ f$$

(b) Draw the source and target syntax trees for the sample string $a^4\,c\,b^5\,c\,a^2\,c\,b^3$ shown above.

(c) Determine if the above translation τ can be defined by means of a finite-state translator T, and provide a suitable explanation for your answer.

---

2. Consider the following control flow graph (*CFG*) of a simple program:



Answer the following questions:

(a) Find the *live variables* of the given *CFG*, by means of the the systematic method of the liveness flow equations: first write the equations, then iteratively solve them. Write the live variables on the *CFG* prepared on the next pages.

(b) Find the *reaching definitions* of the given *CFG*, by intuitively applying the definition of reaching definition. Write the reaching definitions on the *CFG* prepared on the next pages.

(c) (optional) Write the flow equations for the reaching definitions.

---

table of definitions and usages at the nodes for **LIVE VARIABLES**

node	*defined*
1	
2	
3	
4	
5	
6	

node	*used*
1	
2	
3	
4	
5	
6	

system of data-flow equations for **LIVE VARIABLES**

node	*in equations*	*out equations*
1		
2		
3		
4		
5		
6		

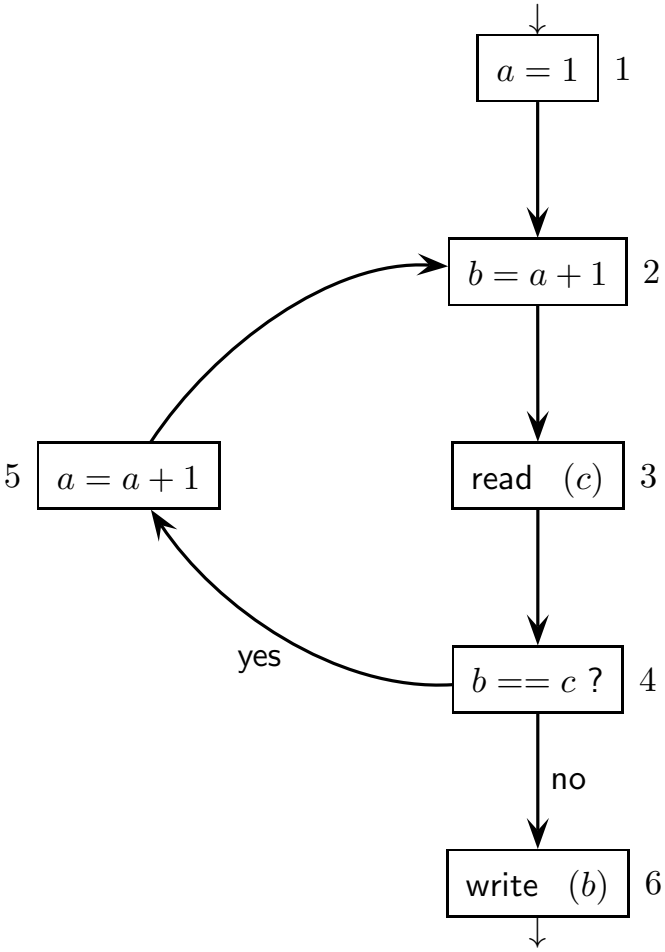
iterative solution table of the system of data-flow equations (**LIVE VAR.S**)

(the number of tables and columns is not significant)

	initialization		1		2		3		4		5		6	
#	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*
1														
2														
3														
4														
5														
6														

	7		8		9		10		11		12		13	
#	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*
1														
2														
3														
4														
5														
6														

please here write the **LIVE VARIABLES** obtained systematically



please here write the **REACHING DEFINITIONS** obtained intuitively

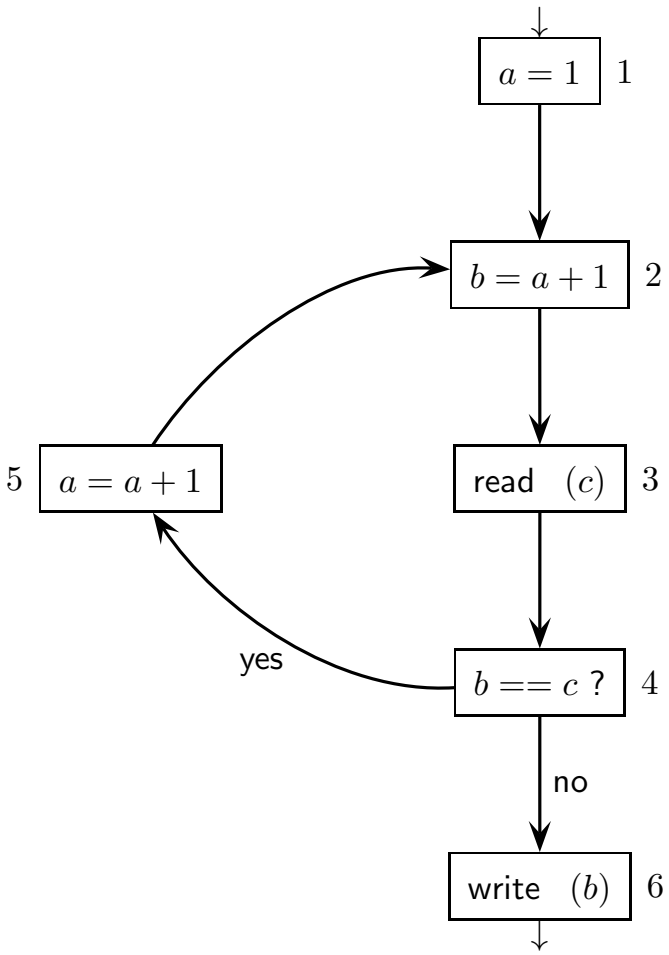


table of definitions and suppressions at the nodes
for **REACHING DEFINITIONS**

node	*defined*
1	
2	
3	
4	
5	
6	

node	*suppressed*
1	
2	
3	
4	
5	
6	

system of data-flow equations for **REACHING DEFINITIONS**

node	*in equations*	*out equations*
1		
2		
3		
4		
5		
6		